

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM : 123240142
123240169
NAMA : RADHITYA RASTA PRAMOJA
ARTSETYA EVAN JUSTINE WIJAYA
PLUG : IF-C
NAMA ASISTEN : Salma Hanifa
Muhammad Irham Hadi Putra

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Radhitya Rasta Pramaja 123240142

Artsetya Evan Justine Wijaya 123240169

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Salma Hanifa
NIM. 123220019

Muhammad Irham Hadi Putra
NIM. 123230042

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Pemilihan Startup untuk Pendanaan Investor Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 25 Mei 2026

Penyusun 1,

Penyusun 2,

Radhitya Rasta Pramoja

NIM. 123240142

Artsetya Evan Justine Wijaya

NIM. 123240169

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	4
2.3 Pemodelan Sistem Pendukung Keputusan	5
2.4 Perhitungan & Algoritma	6
2.5 Implementasi & Tampilan Sistem	6
BAB III JADWAL Pengerjaan dan Pembagian Tugas	21
3.1 Jadwal Pengerjaan	24
3.2 Pembagian Tugas.....	24
BAB IV KESIMPULAN DAN SARAN.....	25
4.2 Kesimpulan.....	25
4.3 Saran	25
DAFTAR PUSTAKA.....	26

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan startup digital di Indonesia semakin meningkat dari tahun ke tahun. Banyak startup bermunculan dengan berbagai inovasi di bidang teknologi, kesehatan, pendidikan, keuangan, dan layanan digital lainnya. Namun, banyaknya pilihan startup membuat investor mengalami kesulitan dalam menentukan startup mana yang paling layak mendapatkan pendanaan.

Setiap startup memiliki keunggulan dan kelemahan yang berbeda, seperti pertumbuhan pendapatan, pertumbuhan pengguna, ukuran pasar, burn rate, dan pengalaman tim. Jika proses pemilihan dilakukan secara manual, maka keputusan dapat menjadi subjektif dan kurang efisien.

Oleh karena itu, dibutuhkan sebuah Sistem Pendukung Keputusan (SPK) yang mampu membantu investor dalam melakukan proses evaluasi startup secara objektif dan terstruktur. Pada proyek akhir ini digunakan metode Simple Additive Weighting (SAW) karena metode ini mampu melakukan perhitungan berdasarkan bobot setiap kriteria dan menghasilkan peringkat startup terbaik.

1.2 Tujuan Proyek Akhir

Tujuan dari proyek akhir ini adalah:

1. Membuat sistem pendukung keputusan untuk pemilihan startup terbaik.
2. Membantu investor dalam menentukan startup yang layak mendapatkan pendanaan.
3. Mengimplementasikan metode Simple Additive Weighting (SAW) pada sistem berbasis Streamlit.
4. Menampilkan hasil peringkat startup berdasarkan nilai akhir SAW.

1.3 Manfaat Proyek Akhir

Manfaat dari proyek akhir ini adalah:

1. Membantu investor mengambil keputusan secara lebih objektif.
2. Mempermudah proses evaluasi startup berdasarkan banyak kriteria.
3. Mengurangi kesalahan pengambilan keputusan secara manual.
4. Memberikan visualisasi data startup secara interaktif.
5. Menjadi media pembelajaran implementasi metode SAW dalam Sistem Pendukung Keputusan.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sistem Pendukung Keputusan (SPK) merupakan sistem berbasis komputer yang digunakan untuk membantu proses pengambilan keputusan dalam menyelesaikan suatu masalah, baik yang bersifat terstruktur maupun semi-terstruktur. SPK membantu pengguna dalam menganalisis data, mempertimbangkan berbagai kriteria, serta menghasilkan keputusan yang lebih objektif dan efisien. Pada proyek ini digunakan metode Simple Additive Weighting (SAW), yaitu metode penjumlahan terbobot yang digunakan untuk menentukan alternatif terbaik berdasarkan beberapa kriteria penilaian. Metode SAW bekerja dengan cara melakukan normalisasi nilai setiap kriteria, kemudian mengalikannya dengan bobot yang telah ditentukan untuk menghasilkan nilai akhir berupa ranking alternatif terbaik.

Tahapan metode SAW :

1. Menentukan kriteria dan alternatif.
2. Menentukan bobot setiap kriteria.
3. Membentuk matriks keputusan.
4. Melakukan normalisasi matriks.
5. Menghitung nilai preferensi.
6. Menentukan peringkat alternatif.

Rumus normalisasi benefit :

$$r_{ij} = \frac{x_{ij}}{\max (x_{ij})}$$

Rumus normalisasi cost :

$$r_{ij} = \frac{\min (x_{ij})}{x_{ij}}$$

Rumus nilai akhir SAW :

$$V_i = \sum_{j=1}^n w_j r_{ij}$$

Keterangan :

- V_i = nilai akhir alternatif
- w_j = bobot kriteria
- r_{ij} = nilai normalisas

2.2 Skenario Kasus & Dataset

Pada proyek ini dilakukan simulasi pemilihan startup terbaik untuk membantu investor dalam menentukan startup yang layak mendapatkan pendanaan. Proses pengambilan keputusan dilakukan dengan mempertimbangkan beberapa kriteria penilaian, seperti total funding, jumlah karyawan, pendapatan tahunan, valuasi, dan skor keberhasilan. Dalam skenario ini, pihak yang bertindak sebagai pengambil keputusan adalah investor atau venture capital yang ingin memilih startup terbaik untuk dijadikan tujuan investasi. Tujuan akhir dari sistem ini adalah menghasilkan rekomendasi startup terbaik berdasarkan hasil perhitungan metode SAW sehingga proses seleksi dapat dilakukan secara lebih objektif, cepat, dan terstruktur.

Dataset yang digunakan pada proyek ini merupakan data startup fiktif yang terdiri dari 5000 data alternatif startup. Setiap data memiliki 5 kriteria penilaian, yaitu total funding, jumlah karyawan, pendapatan tahunan, valuasi, dan skor keberhasilan.

Berikut merupakan contoh sebagian data yang digunakan :

Startup	Total Funding	Jumlah Karyawan	Pendapatan Tahunan	Valuasi	Skor Keberhasilan
Startup_1	269	3047	104	46.11	5
Startup_2	30	630	431	33.04	1
Startup_3	399	2475	375	15.79	8

Dataset digunakan sebagai data simulasi dalam proses perhitungan Sistem Pendukung Keputusan menggunakan metode Simple Additive Weighting (SAW).

2.3 Pemodelan Sistem Pendukung Keputusan

- **Kriteria**

Kriteria	Tipe
Total Funding (Juta USD)	Benefit
Jumlah Karyawan	Benefit
Pendapatan Tahunan (Juta USD)	Benefit
Valuasi (Miliar USD)	Benefit
Skor Keberhasilan	Benefit

- **Alternatif**

Alternatif yang digunakan adalah beberapa startup yang akan dievaluasi.

- **Skala Penilaian**

Nilai	Keterangan
1-20	Sangat Rendah
21-40	Rendah
41-60	Sedang
61-80	Tinggi
81-100	Sangat Tinggi

2.4 Perhitungan & Algoritma

Langkah-langkah algoritma SAW pada sistem:

1. Menginput dataset startup.
2. Menentukan bobot setiap kriteria.
3. Melakukan normalisasi data.
4. Menghitung nilai preferensi menggunakan metode SAW.
5. Mengurutkan nilai terbesar ke terkecil.
6. Menampilkan hasil peringkat startup.

Contoh normalisasi benefit :

$$\frac{80}{92} = 0.869$$

Contoh normalisasi cost :

$$\frac{35}{50} = 0.70$$

Contoh perhitungan nilai akhir:

$$V_i = (0.25 \times 0.869) + (0.20 \times 0.88) + \dots$$

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

# — Konfigurasi Halaman


---


st.set_page_config(
    page_title="SPK Pemilihan Startup - Metode SAW",
    layout="wide",
    initial_sidebar_state="expanded",
)

# — CSS Sederhana


---


st.markdown("""
<style>
    .metric-box {
        background-color: #f0f2f6;

```

```

        border-radius: 8px;
        padding: 16px;
        text-align: center;
        margin: 4px;
    }
    .metric-label { font-size: 13px; color: #555; margin-bottom: 4px; }
    .metric-value { font-size: 26px; font-weight: bold; color: #1f4e79;
}

    .section-title {
        font-size: 18px;
        font-weight: bold;
        color: #1f4e79;
        border-bottom: 2px solid #1f4e79;
        padding-bottom: 4px;
        margin-bottom: 12px;
    }
}
</style>
"""', unsafe_allow_html=True)

# — Load Data


---



@st.cache_data
def load_data():
    df = pd.read_csv("global_startup_success_dataset.csv")
    return df

df_raw = load_data()

KRITERIA = ["Total Funding ($M)", "Number of Employees", "Annual
Revenue ($M)", "Valuation ($B)", "Success Score"]
TIPE      = {
    "Total Funding ($M)"      : "benefit",
    "Number of Employees"    : "benefit",
    "Annual Revenue ($M)"    : "benefit",
    "Valuation ($B)"         : "benefit",
    "Success Score"          : "benefit",
}

NAMA_KRITERIA = {
    "Total Funding ($M)"      : "Total Funding (Juta USD)",
    "Number of Employees"    : "Jumlah Karyawan",
    "Annual Revenue ($M)"    : "Pendapatan Tahunan (Juta USD)",
    "Valuation ($B)"         : "Valuasi (Miliar USD)",
    "Success Score"          : "Skor Keberhasilan",
}

KOLOM_STARTUP = "Startup Name"

# — Sidebar Navigasi


---



st.sidebar.title("SPK Pemilihan Startup")
st.sidebar.markdown("---")
halaman = st.sidebar.radio(
    "Navigasi Halaman",
    ["Beranda", "Dataset", "Hitung SPK", "Visualisasi Analitik",
"Profil Kelompok"],
)
st.sidebar.markdown("---")
st.sidebar.caption("Metode: Simple Additive Weighting (SAW)")

```

```

st.sidebar.caption("Mata Kuliah: Sistem Pendukung Keputusan")

#
=====
# HALAMAN: BERANDA
#
=====

if halaman == "Beranda":
    st.title("Sistem Pendukung Keputusan Pemilihan Startup")
    st.subheader("Metode Simple Additive Weighting (SAW)")
    st.markdown("---")

    col1, col2, col3, col4 = st.columns(4)
    with col1:
        st.markdown(f"""
        <div class="metric-box">
            <div class="metric-label">Total Startup</div>
            <div class="metric-value">{len(df_raw)}</div>
        </div>""", unsafe_allow_html=True)
    with col2:
        st.markdown(f"""
        <div class="metric-box">
            <div class="metric-label">Jumlah Kriteria</div>
            <div class="metric-value">{len(KRITERIA)}</div>
        </div>""", unsafe_allow_html=True)
    with col3:
        st.markdown(f"""
        <div class="metric-box">
            <div class="metric-label">Rata-rata Valuasi</div>
            <div class="metric-value">${df_raw["Valuation
($B)"].mean():.1f}B</div>
        </div>""", unsafe_allow_html=True)
    with col4:
        st.markdown(f"""
        <div class="metric-box">
            <div class="metric-label">Rata-rata Success Score</div>
            <div class="metric-value">{df_raw["Success
Score"].mean():.1f}</div>
        </div>""", unsafe_allow_html=True)

    st.markdown("---")
    st.markdown('<div class="section-title">Tentang Aplikasi
Ini</div>', unsafe_allow_html=True)
    st.markdown("""
    Aplikasi ini merupakan Sistem Pendukung Keputusan (SPK) yang
    dirancang untuk membantu investor
    dalam memilih startup terbaik yang layak mendapatkan pendanaan.

    Metode yang digunakan: Simple Additive Weighting (SAW)

    SAW bekerja dengan cara menormalisasi setiap nilai kriteria
    terhadap nilai terbaik pada kriteria tersebut,
    kemudian mengalikan hasil normalisasi dengan bobot yang diberikan
    oleh investor, dan menjumlahkan
    seluruh nilai untuk mendapatkan skor akhir.
    """)

```

```

st.markdown('<div class="section-title">Kriteria Penilaian</div>',
unsafe_allow_html=True)
tabel_kriteria = pd.DataFrame({
    "Nama Kriteria" : [NAMA_KRITERIA[k] for k in KRITERIA],
    "Kode Kolom" : KRITERIA,
    "Tipe" : [TIPE[k].capitalize() for k in KRITERIA],
    "Keterangan" : [
        "Total pendanaan yang diterima startup dalam juta USD
(semakin besar semakin baik)",
        "Jumlah karyawan yang dimiliki startup (semakin banyak
semakin baik)",
        "Pendapatan tahunan startup dalam juta USD (semakin besar
semakin baik)",
        "Valuasi perusahaan dalam miliar USD (semakin besar semakin
baik)",
        "Skor keberhasilan startup berdasarkan berbagai indikator
1-9 (semakin tinggi semakin baik)",
    ],
})
st.dataframe(tabel_kriteria, use_container_width=True,
hide_index=True)

st.markdown('<div class="section-title">Langkah Penggunaan</div>',
unsafe_allow_html=True)
st.markdown("""
1. Buka halaman Dataset untuk melihat data mentah seluruh
startup.
2. Buka halaman Hitung SPK untuk mengatur bobot kriteria dan
menjalankan perhitungan SAW.
3. Buka halaman Visualisasi Analitik untuk melihat grafik
analitik data startup.
""")

#


---




---


# HALAMAN: DATASET
#


---




---


elif halaman == "Dataset":
    st.title("Dataset Startup")
    st.markdown("---")

    st.markdown('<div class="section-title">Filter & Pencarian
Data</div>', unsafe_allow_html=True)

    col_f1, col_f2 = st.columns(2)
    col_f3, col_f4, col_f5 = st.columns(3)

    with col_f1:
        cari_nama = st.text_input("Cari Nama Startup",
placeholder="Contoh: Startup_1")
    with col_f2:
        pilih_negara = st.multiselect(
            "Filter Negara",
            options=sorted(df_raw["Country"].unique()),
            default=[],

```

```

        placeholder="Semua negara",
    )
    with col_f3:
        pilih_stage = st.multiselect(
            "Filter Funding Stage",
            options=sorted(df_raw["Funding Stage"].unique()),
            default=[],
            placeholder="Semua stage",
        )
    with col_f4:
        pilih_industri = st.multiselect(
            "Filter Industri",
            options=sorted(df_raw["Industry"].unique()),
            default=[],
            placeholder="Semua industri",
        )
    with col_f5:
        min_score = int(df_raw["Success Score"].min())
        max_score = int(df_raw["Success Score"].max())
        score_range = st.slider("Filter Success Score", min_score,
max_score, (min_score, max_score))

    df_filtered = df_raw.copy()
    if cari_nama:
        df_filtered =
df_filtered[df_filtered[KOLOM_STARTUP].str.lower().str.contains(cari_na
ma.lower())]
    if pilih_negara:
        df_filtered =
df_filtered[df_filtered["Country"].isin(pilih_negara)]
    if pilih_stage:
        df_filtered = df_filtered[df_filtered["Funding
Stage"].isin(pilih_stage)]
    if pilih_industri:
        df_filtered =
df_filtered[df_filtered["Industry"].isin(pilih_industri)]
    df_filtered = df_filtered[
        (df_filtered["Success Score"] >= score_range[0]) &
        (df_filtered["Success Score"] <= score_range[1])
    ]

    st.markdown(f"Menampilkan **{len(df_filtered)}** dari
**{len(df_raw)}** data startup")
    st.markdown("----")

    st.markdown('<div class="section-title">Tabel Dataset
Mentah</div>', unsafe_allow_html=True)
    st.dataframe(df_filtered.reset_index(drop=True),
use_container_width=True, hide_index=True)

    st.markdown("----")
    st.markdown('<div class="section-title">Statistik
Deskriptif</div>', unsafe_allow_html=True)
    st.dataframe(df_raw[KRITERIA].describe().round(2),
use_container_width=True)

```

```

#
=====
# HALAMAN: HITUNG SPK
#
=====

elif halaman == "Hitung SPK":
    st.title("Perhitungan SPK - Metode SAW")
    st.markdown("---")

    # — Pengaturan Bobot

    st.markdown('<div class="section-title">Pengaturan Bobot
Kriteria</div>', unsafe_allow_html=True)
    st.info("Atur bobot untuk setiap kriteria. Total bobot akan
dinormalisasi secara otomatis.")

    col_w1, col_w2, col_w3 = st.columns(3)
    col_w4, col_w5, _ = st.columns(3)

    with col_w1:
        w1 = st.number_input("Bobot Total Funding", min_value=0.0,
max_value=100.0,
                                value=20.0, step=1.0, help="Total Funding
($M) - Benefit")
    with col_w2:
        w2 = st.number_input("Bobot Jumlah Karyawan", min_value=0.0,
max_value=100.0,
                                value=15.0, step=1.0, help="Number of
Employees - Benefit")
    with col_w3:
        w3 = st.number_input("Bobot Pendapatan Tahunan", min_value=0.0,
max_value=100.0,
                                value=25.0, step=1.0, help="Annual
Revenue ($M) - Benefit")
    with col_w4:
        w4 = st.number_input("Bobot Valuasi", min_value=0.0,
max_value=100.0,
                                value=25.0, step=1.0, help="Valuation
($B) - Benefit")
    with col_w5:
        w5 = st.number_input("Bobot Success Score", min_value=0.0,
max_value=100.0,
                                value=15.0, step=1.0, help="Success Score
- Benefit")

    bobot_raw = np.array([w1, w2, w3, w4, w5])
    total_bobot = bobot_raw.sum()

    if total_bobot == 0:
        st.error("Total bobot tidak boleh 0. Silakan isi setidaknya
satu bobot.")
        st.stop()

    bobot = bobot_raw / total_bobot

    col_info1, col_info2 = st.columns(2)
    with col_info1:

```

```

df_bobot = pd.DataFrame({
    "Kriteria"          : [NAMA_KRITERIA[k] for k in KRITERIA],
    "Tipe"              : [TIPE[k].capitalize() for k in
KRITERIA],
    "Bobot Input"       : bobot_raw,
    "Bobot Ternormalisasi": [f"{b:.4f} ({b*100:.1f}%)" for b in
bobot],
    })
st.dataframe(df_bobot, use_container_width=True,
hide_index=True)

with col_info2:
    fig_w, ax_w = plt.subplots(figsize=(4, 3))
    ax_w.pie(bobot, labels=[NAMA_KRITERIA[k].split(" ")[0] for k in
KRITERIA],
            autopct="%1.1f%%", startangle=90,
textprops={"fontsize": 8})
    ax_w.set_title("Distribusi Bobot Kriteria", fontsize=10)
    st.pyplot(fig_w)
    plt.close(fig_w)

st.markdown("---")

# — Pemilihan Alternatif

st.markdown('<div class="section-title">Pemilihan Alternatif
Startup</div>', unsafe_allow_html=True)

col_s1, col_s2, col_s3 = st.columns(3)
with col_s1:
    filter_stage = st.multiselect(
        "Filter Funding Stage",
        options=sorted(df_raw["Funding Stage"].unique()),
        default=[],
        placeholder="Semua stage (IPO, Seed, Series A, B, C)",
    )
with col_s2:
    filter_industri_spk = st.multiselect(
        "Filter Industri",
        options=sorted(df_raw["Industry"].unique()),
        default=[],
        placeholder="Semua industri",
    )
with col_s3:
    filter_negara_spk = st.multiselect(
        "Filter Negara",
        options=sorted(df_raw["Country"].unique()),
        default=[],
        placeholder="Semua negara",
    )

df_hitung = df_raw.copy()
if filter_stage:
    df_hitung = df_hitung[df_hitung["Funding
Stage"].isin(filter_stage)]
if filter_industri_spk:
    df_hitung =
df_hitung[df_hitung["Industry"].isin(filter_industri_spk)]
if filter_negara_spk:

```

```

df_hitung =
df_hitung[df_hitung["Country"].isin(filter_negara_spk)]
df_hitung = df_hitung.reset_index(drop=True)

if len(df_hitung) < 2:
    st.warning("Data terlalu sedikit setelah filter. Kurangi filter
agar minimal 2 startup tersedia.")
    st.stop()

st.markdown(f"Startup yang akan dievaluasi: **{len(df_hitung)}
startup**")

pilihan_mode = st.selectbox(
    "Mode Pemilihan Tambahan",
    ["Gunakan Semua Hasil Filter", "Pilih Startup Secara Manual"],
)

if pilihan_mode == "Pilih Startup Secara Manual":
    startup_dipilih = st.multiselect(
        "Pilih Startup yang akan dievaluasi",
        options=df_hitung[KOLOM_STARTUP].tolist(),
        default=df_hitung[KOLOM_STARTUP].tolist()[:10],
    )
    if len(startup_dipilih) < 2:
        st.warning("Pilih minimal 2 startup untuk melakukan
perbandingan.")
        st.stop()
    df_hitung =
df_hitung[df_hitung[KOLOM_STARTUP].isin(startup_dipilih)].reset_index(drop=True)

top_n = st.slider("Tampilkan Top N Startup Terbaik", min_value=5,
max_value=min(50, len(df_hitung)),
                    value=min(10, len(df_hitung)), step=1)

st.markdown("---")

# — Tombol Eksekusi

st.markdown('<div class="section-title">Eksekusi Perhitungan
SAW</div>', unsafe_allow_html=True)

if st.button("Hitung SAW Sekarang", type="primary",
use_container_width=True):
    with st.spinner("Sedang menghitung... harap tunggu."):

        matriks = df_hitung[KRITERIA].values.astype(float)

        # Normalisasi SAW
        matriks_norm = np.zeros_like(matriks)
        for j, k in enumerate(KRITERIA):
            if TIPE[k] == "benefit":
                max_val = matriks[:, j].max()
                matriks_norm[:, j] = matriks[:, j] / max_val if
max_val != 0 else 0
            else: # cost
                min_val = matriks[:, j].min()
                matriks_norm[:, j] = min_val / matriks[:, j] if
min_val != 0 else 0

```

```

# Skor SAW
skor = matriks_norm.dot(bobot)

df_hasil = df_hitung[[KOLOM_STARTUP]].copy()
for j, k in enumerate(KRITERIA):
    df_hasil[k + "_norm"] = matriks_norm[:, j].round(4)
df_hasil["Skor_SAW"] = skor.round(4)
df_hasil = df_hasil.sort_values("Skor_SAW",
ascending=False).reset_index(drop=True)
df_hasil.insert(0, "Peringkat", range(1, len(df_hasil) +
1))

# — Tampilan Hasil


---


st.success("Perhitungan SAW selesai!")
st.markdown("----")

col_r1, col_r2, col_r3 = st.columns(3)
with col_r1:
    st.markdown(f"""
<div class="metric-box">
    <div class="metric-label">Startup Terbaik (Peringkat
1)</div>
    <div class="metric-
value">{df_hasil.iloc[0][KOLOM_STARTUP]}</div>
</div>""", unsafe_allow_html=True)
with col_r2:
    st.markdown(f"""
<div class="metric-box">
    <div class="metric-label">Skor Tertinggi</div>
    <div class="metric-
value">{df_hasil.iloc[0]["Skor_SAW"]:.4f}</div>
</div>""", unsafe_allow_html=True)
with col_r3:
    st.markdown(f"""
<div class="metric-box">
    <div class="metric-label">Total Startup
Dievaluasi</div>
    <div class="metric-value">{len(df_hasil)}</div>
</div>""", unsafe_allow_html=True)

st.markdown("----")
st.markdown(f'<div class="section-title">Peringkat Top {top_n}
Startup Terbaik</div>',
            unsafe_allow_html=True)

df_top = df_hasil.head(top_n)
kolom_tampil = ["Peringkat", KOLOM_STARTUP,
                "Total Funding ($M)_norm", "Number of
Employees_norm",
                "Annual Revenue ($M)_norm", "Valuation
($B)_norm",
                "Success Score_norm", "Skor_SAW"]
rename_map = {
    "Total Funding ($M)_norm" : "Total Funding (N)",
    "Number of Employees_norm" : "Employees (N)",
    "Annual Revenue ($M)_norm" : "Revenue (N)",
    "Valuation ($B)_norm" : "Valuation (N)",

```

```

        "Success Score_norm"          : "Success (N)",
        "Skor_SAW"                   : "Skor SAW",
    }

    # Highlight baris peringkat 1-3
    def warnai(row):
        if row["Peringkat"] == 1:
            return ["background-color: #d4edda"] * len(row)
        elif row["Peringkat"] == 2:
            return ["background-color: #d1ecf1"] * len(row)
        elif row["Peringkat"] == 3:
            return ["background-color: #fff3cd"] * len(row)
        return [""] * len(row)

    df_tampil_top = df_top[kolom_tampil].rename(columns=rename_map)
    st.dataframe(
        df_tampil_top.style.apply(warnai, axis=1),
        use_container_width=True, hide_index=True,
    )

    st.markdown("---")
    st.markdown(f'<div class="section-title">Grafik Peringkat Top
{top_n} Startup</div>',
                unsafe_allow_html=True)

    fig_bar, ax_bar = plt.subplots(figsize=(12, 5))
    colors = ["#1f4e79" if i == 0 else "#2e75b6" if i == 1 else
"#70ad47" if i == 2
            else "#9dc3e6" for i in range(len(df_top))]
    bars = ax_bar.bar(df_top[KOLOM_STARTUP], df_top["Skor_SAW"],
color=colors, edgecolor="white", linewidth=0.5)
    ax_bar.set_title(f"Skor SAW Top {top_n} Startup", fontsize=14,
fontweight="bold")
    ax_bar.set_xlabel("Startup", fontsize=11)
    ax_bar.set_ylabel("Skor SAW", fontsize=11)
    ax_bar.tick_params(axis="x", rotation=45)
    for bar, val in zip(bars, df_top["Skor_SAW"]):
        ax_bar.text(bar.get_x() + bar.get_width() / 2,
bar.get_height() + 0.002,
                    f"{val:.3f}", ha="center", va="bottom",
fontsize=8)
    ax_bar.set_ylim(0, df_top["Skor_SAW"].max() * 1.12)
    plt.tight_layout()
    st.pyplot(fig_bar)
    plt.close(fig_bar)

    st.markdown("---")
    st.markdown('<div class="section-title">Tabel Lengkap Semua
Peringkat</div>',
                unsafe_allow_html=True)
    st.dataframe(df_hasil[["Peringkat", "Startup Name",
"Skor_SAW"]], use_container_width=True, hide_index=True)

    # Simpan ke session state untuk dipakai di halaman lain
    st.session_state["df_hasil"] = df_hasil
    st.session_state["df_top"] = df_top
    st.session_state["top_n"] = top_n

else:

```

```

        st.info("Klik tombol di atas untuk menjalankan perhitungan
SAW.")

#
=====
# HALAMAN: VISUALISASI ANALITIK
#
=====
elif halaman == "Visualisasi Analitik":
    st.title("Visualisasi Analitik Data Startup")
    st.markdown("----")

    # Grafik 1: Distribusi Revenue Growth
    st.markdown('<div class="section-title">1. Distribusi Pertumbuhan
Pendapatan (Revenue Growth)</div>',
                unsafe_allow_html=True)

    fig1, axes1 = plt.subplots(1, 2, figsize=(14, 5))

    sns.histplot(df_raw["Annual Revenue ($M)"], bins=20, kde=True,
color="#2e75b6",
                ax=axes1[0], edgecolor="white")
    axes1[0].set_title("Histogram Annual Revenue", fontweight="bold")
    axes1[0].set_xlabel("Annual Revenue ($M)")
    axes1[0].set_ylabel("Jumlah Startup")
    axes1[0].axvline(df_raw["Annual Revenue ($M)"].mean(), color="red",
linestyle="--",
                    label=f"Rata-rata: {df_raw['Annual Revenue
($M)'].mean():.1f}%")
    axes1[0].axvline(df_raw["Annual Revenue ($M)"].median(),
color="orange", linestyle="-. ",
                    label=f"Median: {df_raw['Annual Revenue
($M)'].median():.1f}%")
    axes1[0].legend()

    sns.boxplot(data=df_raw[KRITERIA], ax=axes1[1], palette="Blues")
    axes1[1].set_title("Boxplot Semua Kriteria (Nilai Asli)",
fontweight="bold")
    axes1[1].set_xticklabels([NAMA_KRITERIA[k].split(" ")[0] for k in
KRITERIA], rotation=20, ha="right")
    axes1[1].set_ylabel("Nilai")

    plt.tight_layout()
    st.pyplot(fig1)
    plt.close(fig1)

    st.markdown("Grafik di atas menunjukkan distribusi Revenue Growth
seluruh startup beserta persebaran nilai tiap kriteria.")
    st.markdown("----")

    # Grafik 2: Heatmap Korelasi
    st.markdown('<div class="section-title">2. Heatmap Korelasi Antar
Kriteria</div>', unsafe_allow_html=True)

    fig2, ax2 = plt.subplots(figsize=(8, 6))
    corr = df_raw[KRITERIA].corr()

```

```

mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            mask=mask, ax=ax2, linewidths=0.5, square=True,
            xticklabels=[NAMA_KRITERIA[k].split(" ")[0] for k in
KRITERIA],
            yticklabels=[NAMA_KRITERIA[k].split(" ")[0] for k in
KRITERIA])
ax2.set_title("Matriks Korelasi Antar Kriteria", fontweight="bold",
fontsize=13)
plt.tight_layout()
st.pyplot(fig2)
plt.close(fig2)

st.markdown("Heatmap menunjukkan kekuatan hubungan antar kriteria.
Nilai mendekati 1 atau -1 menandakan korelasi kuat.")
st.markdown("----")

# Grafik 3: Scatter Plot Revenue Growth vs User Growth
st.markdown('<div class="section-title">3. Scatter Plot: Revenue
Growth vs User Growth</div>',
            unsafe_allow_html=True)

pilih_x = st.selectbox("Pilih Sumbu X", KRITERIA, index=0)
pilih_y = st.selectbox("Pilih Sumbu Y", KRITERIA, index=1)
pilih_warna = st.selectbox("Warna berdasarkan", KRITERIA, index=4)

fig3, ax3 = plt.subplots(figsize=(12, 6))
sc = ax3.scatter(df_raw[pilih_x], df_raw[pilih_y],
                c=df_raw[pilih_warna], cmap="viridis", alpha=0.7,
s=80, edgecolors="white", linewidth=0.5)
plt.colorbar(sc, ax=ax3, label=NAMA_KRITERIA[pilih_warna])
ax3.set_xlabel(NAMA_KRITERIA[pilih_x], fontsize=11)
ax3.set_ylabel(NAMA_KRITERIA[pilih_y], fontsize=11)
ax3.set_title(f"Scatter: {NAMA_KRITERIA[pilih_x]} vs
{NAMA_KRITERIA[pilih_y]}", fontweight="bold")

# Tambahkan label untuk beberapa startup dengan nilai ekstrem
threshold_x = df_raw[pilih_x].quantile(0.95)
threshold_y = df_raw[pilih_y].quantile(0.95)
for _, row in df_raw.iterrows():
    if row[pilih_x] >= threshold_x or row[pilih_y] >= threshold_y:
        ax3.annotate(row["Startup Name"], (row[pilih_x],
row[pilih_y]),
                    textcoords="offset points", xytext=(5, 5),
                    fontsize=7, color="#1f4e79")

plt.tight_layout()
st.pyplot(fig3)
plt.close(fig3)

st.markdown("----")

# Grafik 4: Bar Chart Top 15 Startup per Kriteria
st.markdown('<div class="section-title">4. Top 15 Startup per
Kriteria</div>', unsafe_allow_html=True)

pilih_kriteria = st.selectbox("Pilih Kriteria untuk Ditampilkan",
KRITERIA,

```

```

format_func=lambda x:
NAMA_KRITERIA[x])
    ascending = TIPE[pilih_kriteria] == "cost"
    df_top15 = df_raw.nlargest(15, pilih_kriteria) if not ascending
else df_raw.nsmallest(15, pilih_kriteria)
    df_top15 = df_top15.sort_values(pilih_kriteria,
ascending=ascending)

    fig4, ax4 = plt.subplots(figsize=(12, 6))
    colors4 = plt.cm.Blues(np.linspace(0.4, 0.9, len(df_top15)))
    if not ascending:
        colors4 = plt.cm.Blues(np.linspace(0.9, 0.4, len(df_top15)))

    bars4 = ax4.barh(df_top15["Startup Name"],
df_top15[pilih_kriteria], color=colors4, edgecolor="white")
    ax4.set_title(f"Top 15 Startup - {NAMA_KRITERIA[pilih_kriteria]}",
fontweight="bold", fontsize=13)
    ax4.set_xlabel(NAMA_KRITERIA[pilih_kriteria], fontsize=11)

    for bar, val in zip(bars4, df_top15[pilih_kriteria]):
        ax4.text(bar.get_width() + bar.get_width() * 0.01, bar.get_y()
+ bar.get_height() / 2,
                f"{val:,.0f}", va="center", fontsize=9)

    ax4.invert_yaxis()
    plt.tight_layout()
    st.pyplot(fig4)
    plt.close(fig4)

    st.markdown("----")

    # Grafik 5: Radar Chart Top 5 Startup (dari hasil SAW jika ada)
    st.markdown('<div class="section-title">5. Radar Chart Perbandingan
Startup</div>', unsafe_allow_html=True)

    if "df_hasil" in st.session_state:
        df_radar_base = st.session_state["df_hasil"].merge(df_raw,
on="Startup Name")
        startup_pilih_radar = df_radar_base["Startup
Name"].head(5).tolist()
    else:
        startup_pilih_radar = df_raw["Startup Name"].head(5).tolist()
        df_radar_base = df_raw.copy()

    startup_radar = st.multiselect("Pilih Startup untuk Radar Chart
(maks 6)",
                                df_raw["Startup Name"].tolist(),
                                default=startup_pilih_radar[:5],
                                key="radar_ms")

    if len(startup_radar) < 2:
        st.warning("Pilih minimal 2 startup untuk radar chart.")
    else:
        df_norm_all = df_raw.copy()
        for k in KRITERIA:
            if TIPE[k] == "benefit":
                df_norm_all[k] = df_raw[k] / df_raw[k].max()
            else:
                df_norm_all[k] = df_raw[k].min() / df_raw[k]

```

```

        df_radar_plot = df_norm_all[df_norm_all["Startup
Name"].isin(startup_radar)]
        kategoris = [NAMA_KRITERIA[k].split(" ")[0] for k in KRITERIA]
        N = len(kategoris)
        angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()
        angles += angles[:1]

        fig5, ax5 = plt.subplots(figsize=(8, 8),
subplot_kw=dict(polar=True))
        palette = plt.cm.tab10.colors

        for idx, (_, row) in enumerate(df_radar_plot.iterrows()):
            values = [row[k] for k in KRITERIA]
            values += values[:1]
            color = palette[idx % 10]
            ax5.plot(angles, values, color=color, linewidth=2,
label=row["Startup Name"])
            ax5.fill(angles, values, color=color, alpha=0.15)

            ax5.set_xticks(angles[:-1])
            ax5.set_xticklabels(kategoris, fontsize=11)
            ax5.set_ylim(0, 1.05)
            ax5.set_title("Radar Chart Perbandingan Startup (Nilai
Ternormalisasi)", fontweight="bold",
                        fontsize=12, pad=20)
            ax5.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1),
fontsize=10)
            plt.tight_layout()
            st.pyplot(fig5)
            plt.close(fig5)

        st.markdown("---")

#
=====
=====
# HALAMAN: PROFIL KELOMPOK
#
=====
=====
elif halaman == "Profil Kelompok":
    st.title("Profil Kelompok")
    st.markdown("---")

    st.markdown('<div class="section-title">Identitas Kelompok</div>',
unsafe_allow_html=True)
    st.markdown("""
**Mata Kuliah:** Sistem Pendukung Keputusan

**Topik Proyek:** Pemilihan Startup untuk Membantu Investor dalam
Pendanaan dengan Metode SAW
""")

    st.markdown("---")
    st.markdown('<div class="section-title">Anggota Kelompok</div>',
unsafe_allow_html=True)

```

```

col_a, col_b = st.columns(2)

with col_a:
    st.markdown("""
        **Anggota 1**

        Nama : Radhitya Rasta Pramoja

        NIM  : 123240142
        """)

with col_b:
    st.markdown("""
        **Anggota 2**

        Nama : Artsetya Evan Justine Wijaya

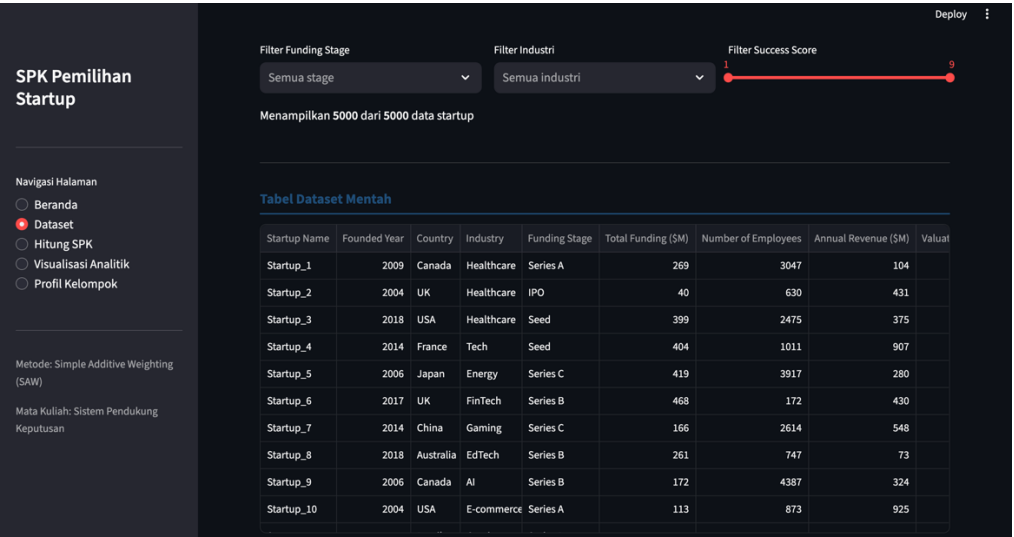
        NIM  : 123240169
        """)

st.markdown("---")
st.markdown('<div class="section-title">Dataset yang
Digunakan</div>', unsafe_allow_html=True)
st.markdown(f"""
- Nama file      : DataStartup.csv
- Jumlah data   : {len(df_raw)} startup
- Jumlah kolom  : {len(df_raw.columns)} (1 identitas + 5 kriteria)
- Kriteria      : Revenue Growth, User Growth, Market Size, Burn
Rate, Team Experience
""")

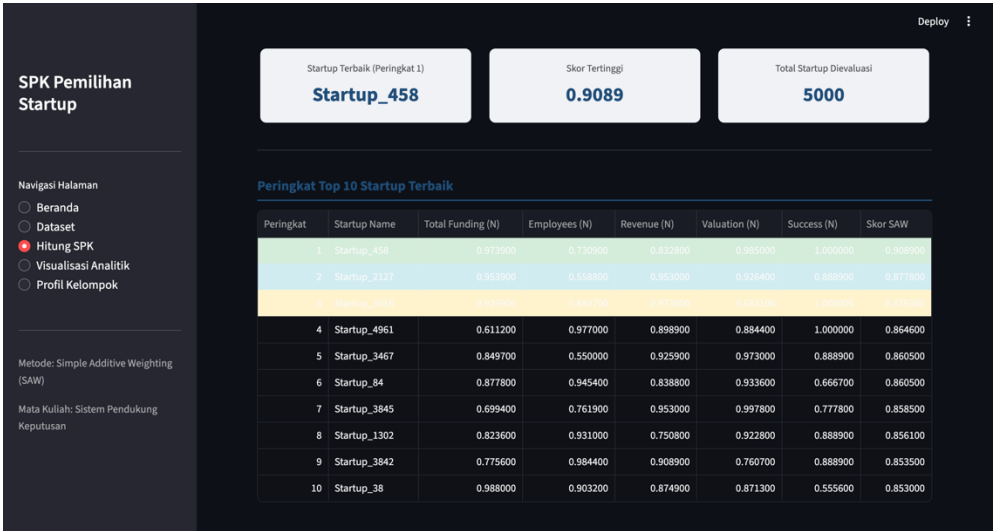
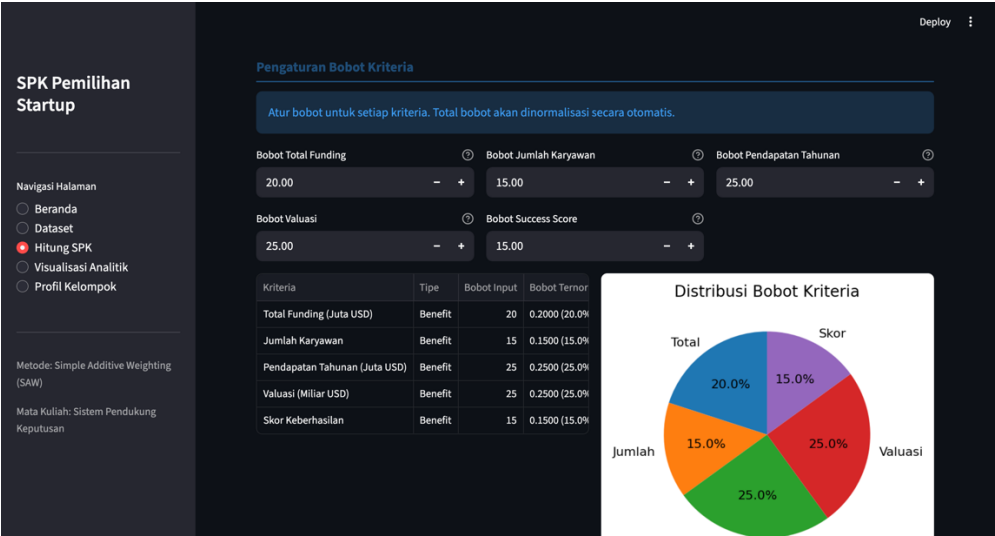
```



Halaman Beranda

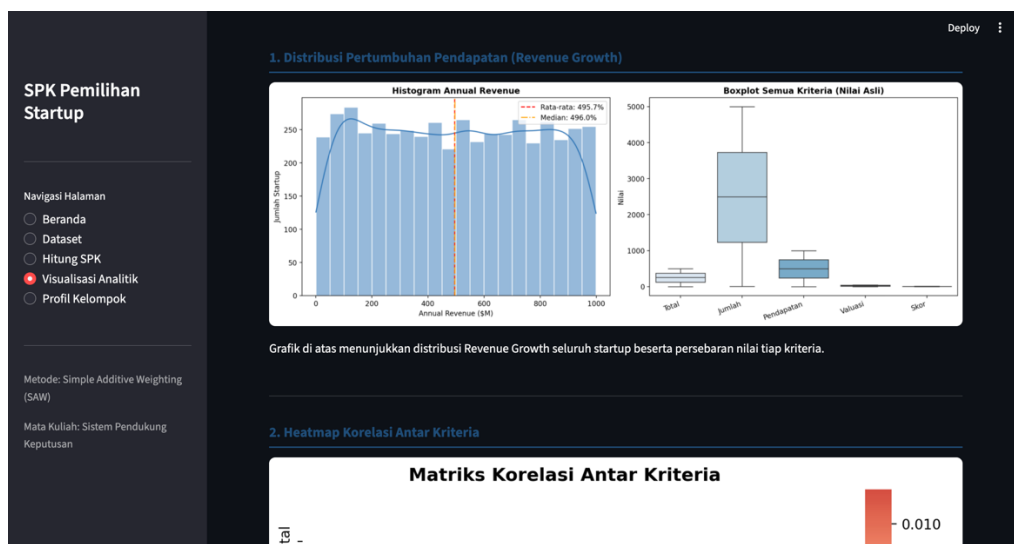


Halaman Dataset





Halaman Hasil SAW



Halaman Visualisasi Analitik

BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2022			
		Minggu			
		1	2	3	4
1	Analisis Masalah	✓			
2	Pengumpulan Dataset		✓		
3	Pembuatan Sistem			✓	✓
4	Penyusunan Laporan				✓

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Radhitya Rasta Pramoja	123240142	Membuat dataset, dokumentasi sistem, pembuatan laporan
2	Artsetya Evan Justine Wijaya	123240169	Membuat program SAW, visualisasi data, pengujian program

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil proyek akhir yang telah dibuat, metode Simple Additive Weighting (SAW) dapat digunakan untuk membantu proses pemilihan startup terbaik secara objektif berdasarkan beberapa kriteria penilaian.

Sistem yang dibuat mampu melakukan normalisasi data, perhitungan nilai akhir, serta menghasilkan ranking startup terbaik. Selain itu, sistem juga dilengkapi dengan visualisasi data sehingga memudahkan pengguna dalam melakukan analisis.

4.3 Saran

Saran untuk pengembangan sistem ke depannya:

1. Menambahkan metode SPK lain seperti AHP atau TOPSIS.
2. Menggunakan dataset real-time dari internet.
3. Menambahkan fitur login dan penyimpanan database.
4. Mengembangkan sistem berbasis web deployment online.

DAFTAR PUSTAKA

https://github.com/Wijaya-23/Project_Akhir_Prak_SCPK